Web Development — Lecture Notes

Unit-wise concise notes

Lecture #1 — Introduction to Web Development (Important)

1. Course Introduction, CLOs, Assessment & Grading

This course introduces the foundations of modern web development and the technologies used to build web applications. Course Learning Outcomes (CLOs) describe what students should be able to understand or implement by the end of the course. The assessment plan may include quizzes, assignments, labs, projects, midterm/final examinations and viva. The grading policy explains how marks from these activities contribute to the final grade.

2. Internet vs. Web

The Internet is the global network infrastructure that connects computers and devices. The World Wide Web (Web) is one service that runs over the Internet and provides linked pages and applications that users access mainly through web browsers. Other Internet services include email, file transfer and messaging.

3. Client–Server Request–Response Model

A client, usually a browser, requests a resource or service from a server. The server receives and processes the request and sends a response back to the client. For example, entering a website address in a browser can cause the browser to send an HTTP request; the web server then returns HTML, CSS, JavaScript, data or another resource.

4. Core Web Terminologies

Term	Short Note	Example
URL / URI	A URI identifies a resource; a URL is a type of URI that also tells how and where the resource can be accessed.	Example URL: https://bookstore.com/books. The URL identifies the resource and how to access it using HTTPS. **URI: Urn:Msdn:1232140987** **URL: https://bookstore.com/books**
HTTP	Hypertext Transfer Protocol defines how web clients and servers exchange requests and responses.	Example: Browser sends GET /books to the web server to request the books page or data. Examples from Apps App Transport Security (ATS) for **iOS** Network Security Configuration (NSC) for **Android**
HTTPS	The secure form of HTTP. It uses encryption through TLS to protect data exchanged between the browser and server.	Example: https://bookstore.com uses HTTPS so data between the browser and server is encrypted using TLS. What if we use bare https://159 within app ? TLS (Transport Layer Security) Old SSL Certificate (Still commonly used name)
DNS	Domain Name System translates human-friendly domain names, such as example.com, into IP addresses used by computers.	Example: bookstore.com can have A record that resolves to 159.65.157.115, allowing the client to identify the server IP to contact.
Hosting	A service that provides server resources and storage so a website or web application can be available on the Internet.	Example: A React/Node website may be hosted on a DigitalOcean Droplet so it is accessible over the Internet.
Web Server	Software or a computer system that receives web requests and returns web resources or application responses.	Example: Nginx on 159.65.157.115 can receive a request for bookstore.com and return the requested resource or forward it to a Node.js application. **Nginx**
Browser	Client software such as Chrome, Firefox, Safari or Edge that requests, interprets and displays web content.	Example: Chrome receives HTML, CSS, JavaScript, images or JSON through HTTP/HTTPS and displays or processes them for the user.

5. Web Standards and Standards Bodies

- **W3C:** Develops and promotes standards and guidelines for an open and accessible Web, including technologies such as CSS and accessibility guidance.
- **WHATWG:** Maintains the living HTML and DOM standards used by modern browsers.
- **ECMA International:** Standardizes ECMAScript, the language specification on which JavaScript is based.
- **IETF:** Develops Internet standards and protocols through documents called RFCs; its work includes important web and networking protocols.

6. Types of Web Applications

- **Static Website:** Pages are generally delivered as stored files and usually show the same prepared content until the files are changed.
- **Dynamic Website:** Content can be generated or changed using application logic, databases, user input or other data sources.
- **MPA — Multi-Page Application:** The application consists of multiple pages; navigation commonly loads another page or document from the server.
- **SPA — Single-Page Application:** A main page is loaded and JavaScript updates the interface dynamically, often without full-page reloads. ReactJS is commonly used to build SPAs.
- **PWA — Progressive Web App:** A web application enhanced with modern browser capabilities so it can provide app-like features such as installability and, when designed for it, offline support.

Lecture #1 — Quick Revision

- Internet = global network infrastructure; Web = a service that uses the Internet.
- Client sends a request; server processes it and returns a response.
- DNS resolves domain names; HTTP/HTTPS carries web requests and responses.
- HTTPS adds encrypted communication using TLS.
- W3C, WHATWG, ECMA International and IETF contribute standards used across the Web and Internet.
- Common application types include static, dynamic, MPA, SPA and PWA.

Viva / Concept Check:

Lecture #2 — Tiered Web Architecture

1. Tiers vs. Layers; Separation of Concerns (Modular Approach, Component Based S/w)

A tier is a physically or independently deployed part of an application, while a layer is a logical division of responsibilities inside the software. For example, presentation, business logic and data access can be separate layers even when they run on the same server. Separation of concerns means giving each part of the system a clear responsibility so that the application is easier to understand, test, maintain and modify.

Concept	Meaning	Example
Layer	Logical separation of responsibilities.	UI layer, business-logic layer, data-access layer.
Tier	Physical/deployment separation that can run on a different process, machine or server.	React client on a user device, Node.js API on a server, MongoDB on a database server.

2. One-Tier (Standalone) Architecture

In one-tier architecture, the user interface, application logic and data are kept together in one application or system. The application can work without communicating with a separate application server for its main processing.

Example: A simple desktop calculator performs its user interface and calculation logic locally. Another example is a small offline desktop application that stores its data in a local file or local database.

Quran Apps which are running without Server are examples of One-Tier Architecture (**Asan Quran**)

Simple flow: User → Standalone Application → Local Data/Processing

3. Two-Tier (Client–Server) Architecture

In two-tier architecture, the application is divided into a client tier and a server/data tier. The client handles the user interface and may contain some application logic, while it communicates directly with the server or database for data and services.

Example: A desktop inventory application directly connects to a database server to read and update product records. In a simple client-server system, the client sends requests and the server returns the requested data or service.

Simple flow: Client Application ↔ Server / Database (**PDF BookStore**)

- Advantages: simpler than multi-tier systems, relatively easy to develop for small systems, and direct communication can reduce architectural complexity.
- Limitations: direct client-to-database coupling can make security, maintenance, scaling and deployment harder as the number of clients and application requirements grow.

4. Three-Tier Architecture

Three-tier architecture separates the system into presentation, application/business-logic and data tiers. The client normally communicates with the application server instead of accessing the database directly.

MVC (Model View Controller)

Tier	Responsibility	Web Example
Presentation Tier		ReactJS application in the browser.

	Displays the interface and receives user interaction.	
Application / Logic Tier	Processes requests, applies business rules and exposes APIs.	Node.js + Express API.
Data Tier	Stores and retrieves persistent application data.	MongoDB database.

Example flow: ReactJS Client → HTTP/HTTPS API → **Node.js/Express → MongoDB** → Response back to ReactJS.

5. N-Tier Architecture

N-tier architecture extends the same idea by separating the application into more independently managed tiers when the system becomes larger. For example, a system may have a web/client tier, API tier, authentication service, application services, cache and database tier. More tiers can improve separation and independent scaling, but they also add deployment, networking and operational complexity.

6. Selecting an Architecture

Factor	Question to Ask	Trade-off
Scalability	How many users, requests and data operations must the system support?	More separated tiers can be scaled independently, but require more infrastructure.
Maintainability	Will different parts of the system change frequently or be maintained by different teams?	Clear separation can make changes safer and easier, but introduces more components.
Cost	What infrastructure, development and operational budget is available?	A simple architecture costs less to build and operate; additional tiers can increase servers, services and management cost.

There is no single architecture that is best for every application. A small standalone tool may need only one tier, while a large web application may benefit from three-tier or N-tier architecture. The choice should match the application's size, security needs, expected growth, maintainability requirements and available budget.

Lecture #2 — Quick Revision

- Tier = deployment/physical separation; layer = logical separation of responsibilities.
- Separation of concerns gives different parts of the application clear responsibilities.
- One-tier keeps the main application components together in a standalone system.
- Two-tier separates client and server/data responsibilities.
- Three-tier commonly separates presentation, application logic and data.
- N-tier adds further separation when system requirements justify the extra complexity.
- Architecture selection involves scalability, maintainability, security, complexity and cost trade-offs.

Viva / Concept Check

Lecture #3 — HTML and HTML5 Fundamentals (Important)

1. Role of Markup and HTML Document Structure

HTML (HyperText Markup Language) is the standard markup language used to describe the structure and meaning of content on a web page. HTML is not a programming language; markup tells the browser what content represents, such as a heading, paragraph, image, link, table or list.

Basic HTML document structure:

```
<!DOCTYPE html>

<html>

  <head>

    <title>Book Store</title>

   Import css file

  </head>

  <body>

    <h1>Welcome to Book Store</h1> h1, h2 h3 h4 h5 h6

    <p >Browse our available books.</p>

  </body>

</html>
```

- DOCTYPE: <!DOCTYPE html> tells the browser that the document uses modern HTML.
- Head: Contains metadata and resources such as the page title, meta information, stylesheet links and scripts that belong in the document head.
- Body: Contains the page content normally displayed to the user, such as headings, paragraphs, links, images, tables and forms.

2. Elements, Tags, Attributes, Nesting and Validation

An HTML element represents a part of the document. Many elements use an opening tag, content and a closing tag. Attributes provide additional information or configuration for an element.

Concept	Meaning	Example
Element	The complete HTML structure for a piece of content.	Hello Students
Tag	Markup syntax that starts or ends an element.	<p classname= > and
Attribute	Additional information written in a start tag.	Books
Nesting	Placing one element correctly inside another.	Book details
Validation	Checking HTML against the standard to find markup errors.	Missing closing tags or invalid nesting can be detected by a validator.

Correct nesting example: Hello . Elements should be opened and closed in a properly nested order. HTML validation is useful for finding structural mistakes and improving standards compliance.

3. Headings, Paragraphs, Text Formatting and Hyperlinks

- Headings: <h1> to <h6> define headings of decreasing importance. <h1> is normally used for the main heading of a page.
- Paragraph: defines a paragraph of text.
- Text meaning/formatting: Common elements include for strong importance and for emphasis. Elements such as <b> and <i> are also available but have different semantic purposes.
- Hyperlink: <a> creates a link. Its href attribute specifies the destination.

Absolute URL example: Books — contains a complete URL including scheme and domain.

Relative URL example: Books — points to a resource relative to the current document/base URL.

4. Images, Media, Tables and Lists

Feature	HTML Example	Purpose
Images	<img src="images/quran-cover.jpg" alt="Quran book cover">	src identifies the image resource; alt provides alternative text.
Audio	<audio controls src="audio/lesson.mp3"></audio>	Embeds audio content with browser controls.
Video	<video controls src="video/intro.mp4"></video>	Embeds video content with browser controls.
Table	<table>, <tr>, <th>, <td>	Used for tabular data arranged in rows and columns.
Unordered List	- HTML - CSS	Displays a list where order is not essential.
Ordered List	- HTML - CSS	Displays an ordered/numbered sequence.

5. HTML5 Elements, Attributes and Browser Support

HTML5 introduced and popularized semantic elements and built-in capabilities that make documents easier to structure and reduce dependence on older plugin-based approaches. Modern HTML continues as a living standard.

- <header> — introductory/header content for a page or section.
- <nav> — a section containing major navigation links.
- <main> — the dominant/main content of the document.
- <section> — a thematic section of content.
- <article> — self-contained content such as a post or article.
- <aside> — related or supplementary content.
- <footer> — footer information for a page or section.
- <audio> and <video> — native media elements.
- Useful form-related features include input types such as email, date and number, and attributes such as placeholder and required.

Browser support: Modern browsers support the major HTML5 features, but support for particular elements, attributes or APIs can vary by browser and version. Developers should check compatibility when using newer or less common features and provide suitable fallbacks when necessary.

Lecture #3 — Quick Revision

- HTML provides the structure and meaning of web-page content.
- <!DOCTYPE html>, <head> and <body> are fundamental parts of an HTML document.
- Elements are written using tags; attributes add information to elements; correct nesting keeps document structure valid.
- Absolute links contain a complete URL, while relative links are resolved relative to the current document/base URL.
- HTML supports images, audio, video, tables, ordered lists and unordered lists.
- HTML5 provides semantic elements such as header, nav, main, section, article and footer, along with modern media and form capabilities.
- Browser compatibility should be considered when using newer web features.

Viva / Concept Check:

Lecture #4 — Semantic HTML and HTML Forms (Important)

1. Block-Level vs. Inline Elements; div and span

HTML elements differ in how they normally participate in page layout. Block-level elements usually begin on a new line and occupy the available horizontal space, while inline elements remain within the current line and take only the space required by their content.

Concept	Meaning	Example
Block-level element	Normally starts on a new line and forms a block in the document flow.	, <p>, <section>, <header>
Inline element	Normally stays within the current line and occupies only the required width.	, <a>, ,
	A generic block container used to group content when no more meaningful semantic element is suitable.	...
	A generic inline container used to group or style a small part of text.	Rs. 500

Example:

```
<div>This is a block container</div>

<p> Price: <span>Rs. 500</span> </p>
```

A div and span do not describe the meaning of their content. They are useful generic containers, but semantic elements should be preferred when the content has a clear purpose.

2. Semantic HTML Elements

Semantic HTML uses elements whose names describe the role or meaning of the content. This makes the document structure easier for developers, browsers, search engines and assistive technologies to understand.

Element	Typical Purpose	Example Use
<header>	Introductory content for a page or section.	Site title, logo, introductory heading
<nav>	Major navigation links.	Home, Books, Contact links
<main>	The dominant content of the page.	Main bookstore content
<section>	A thematic grouping of related content.	Featured Books section
<article>	Self-contained content that can stand independently.	Book review or blog post
<aside>	Related or supplementary content.	Related books or side information
<footer>	Footer information for a page or section.	Copyright and contact information

Simple semantic structure:

```
<header>...</header>

<nav>...</nav>

<main>
```

```
  <section>
    <article>...</article>
  </section>
  <aside>...</aside>
</main>
<footer>...</footer>
```

3. Accessibility and SEO Benefits

- Accessibility: Semantic structure helps assistive technologies understand page regions and navigation.
- SEO: Meaningful structure can help search engines interpret the purpose and hierarchy of page content.
- Maintainability: Descriptive elements make source code easier for developers to read and modify.
- Consistency: Using elements according to their intended meaning produces a clearer document structure.

Semantic HTML does not automatically guarantee accessibility or a high search ranking. Correct headings, labels, alternative text, keyboard support and meaningful content are also important.

4. HTML Forms: action and method

A form collects user input and submits it for processing. The <form> element commonly uses the action and method attributes.

Attribute	Purpose	Example
action	Specifies where the submitted form data should be sent.	action="/register"
method	Specifies the HTTP method used to submit the form, commonly GET or POST.	method="post"

```
<form action="/register" method="post">
  ... form controls ...
</form>
```

GET commonly places submitted values in the URL query string and is suitable for operations such as searches. POST sends submitted data in the request body and is commonly used when data is created or submitted for processing.

5. Common Form Controls

Control	Purpose	Example
<input>	Single-line input; its type controls the kind of value expected.	<input type="email" name="email">
<select>	Provides a list of options.	<select name="city">...</select>
<textarea>	Accepts multi-line text.	<textarea name="message"></textarea>
<button>	Creates a clickable button, often for form submission.	<button type="submit">Submit</button>

Common input types include text, password, email, number, date, radio, checkbox and submit. A <label> should normally be associated with a form control so users can understand what information is required.

6. HTML5 Client-Side Validation

HTML5 provides validation attributes that allow the browser to check common input rules before form submission. This improves user feedback, but server-side validation is still required because client-side checks can be bypassed.

Attribute	Purpose	Example
required	Makes a value mandatory.	<input type="email" required>
pattern	Requires text to match a regular-expression pattern.	<input type="text" pattern="[A-Za-z]+">
min	Specifies the minimum permitted value.	<input type="number" min="1">
max	Specifies the maximum permitted value.	<input type="number" max="100">

```
<input type="number" name="age" min="18" max="60" required>
```

Quick Revision

- Block elements normally form blocks in document flow; inline elements remain within a line.
- div is a generic block container; span is a generic inline container.
- Semantic elements describe the purpose of page regions and improve document structure.
- Forms collect user input; action identifies the destination and method identifies how data is submitted.
- Common controls include input, select, textarea and button.
- required, pattern, min and max provide browser-side validation, but server-side validation is still necessary.

Viva / Concept Check

1. What is the difference between a block-level and an inline element?

2. Why should semantic HTML be preferred over using div for every section?

3. What is the difference between GET and POST in an HTML form?

4. What is the purpose of the required and pattern attributes?

5. Why is client-side validation not sufficient by itself?

Lecture #5 — Cascading Style Sheets (CSS) Fundamentals (Important)

1. CSS Syntax

CSS (Cascading Style Sheets) controls the presentation of HTML content, including colours, fonts, spacing, backgrounds and layout. A CSS rule normally contains a selector and a declaration block.

```
selector {
  property: value;
}
```

```
p { color: blue; font-size: 16px; }
```

In this example, p is the selector, color and font-size are properties, and blue and 16px are their values.

2. Inline, Internal and External CSS

Type	Where CSS is Written	Example / Use
Inline	Inside an HTML element using the style attribute.	Hello
Internal	Inside a <style> element, normally in the document head.	<style> p { color:red; } </style>
External	In a separate .css file linked to the HTML document.	<link rel="stylesheet" href="styles.css">

External CSS is generally preferred for multi-page sites because the same stylesheet can be reused and maintained separately from the HTML.

3. CSS Selectors

Selector	Meaning	Example
Element	Selects elements by tag name.	p { ... }
Class	Selects elements having a class.	.book { ... }
ID	Selects the element with a particular id.	#header { ... }
Attribute	Selects elements according to an attribute.	input[type="email"] { ... }
Grouping	Applies the same declarations to several selectors.	h1, h2, p { ... }
Descendant	Selects matching elements inside another element.	nav a { ... }

4. Pseudo-Classes and Pseudo-Elements

A pseudo-class selects an element in a particular state or condition. A pseudo-element selects or creates a specific part of an element.

Type	Example	Meaning
Pseudo-class	a:hover	

		Selects a link while the pointer is over it.
Pseudo-class	input:focus	Selects an input while it has focus.
Pseudo-class	li:first-child	Selects a list item when it is the first child.
Pseudo-element	p::first-letter	Selects the first letter of a paragraph.
Pseudo-element	.title::before	Creates/selects generated content before an element's content.

5. Cascade, Specificity and Inheritance

The word cascading refers to the process CSS uses to determine which declaration applies when more than one rule targets the same element. Important factors include the source of a rule, importance, specificity and source order.

- Specificity: More specific selectors normally take precedence over less specific selectors when competing declarations have the same importance and origin.
- Source order: When competing rules have the same specificity, a later applicable rule can override an earlier one.
- Inheritance: Some properties, such as many text-related properties, can be inherited from a parent element by its children.

```
p { color: blue; }

.important { color: red; }
```

For Text , the class selector is more specific than the element selector, so red normally wins for the color property.

6. Colour, Font, Text and Background Properties

Category	Common Properties	Example
Colour	color	color: #333;
Font	font-family, font-size, font-weight, font-style	font-size: 18px;
Text	text-align, text-decoration, line-height	text-align: center;
Background	background-color, background-image, background-size	background-color: #f5f5f5;

CSS colours can be expressed using named colours, hexadecimal values, rgb()/rgba() and other supported colour formats.

Quick Revision

- A CSS rule contains a selector and declarations written as property-value pairs.
- CSS can be inline, internal or external; external stylesheets are reusable across pages.
- Common selectors include element, class, id, attribute, grouping and descendant selectors.
- Pseudo-classes represent states or conditions; pseudo-elements target parts of elements.
- The cascade resolves competing rules using factors such as specificity and source order.
- CSS controls colours, typography, text presentation and backgrounds.

Viva / Concept Check

1. What are the three ways to apply CSS to HTML?

2. What is the difference between a class selector and an id selector?

3. What is a pseudo-class? Give one example.

4. What is CSS specificity?

5. What does inheritance mean in CSS?

Lecture #6 — The CSS Box Model and Display (Important)

1. CSS Box Model

Every rendered HTML element can be considered a rectangular box. From the inside outward, the box model consists of content, padding, border and margin.

Part	Meaning
Content	The actual text, image or other content of the element.
Padding	Space between the content and the border.
Border	The line or edge surrounding the padding and content.
Margin	Space outside the border that separates the element from neighbouring elements.

```
.card {
  width: 300px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
}
```

2. Margin Collapsing

Vertical margins of adjacent block elements can sometimes collapse so that they combine into a single margin rather than being added together. This behaviour mainly concerns vertical margins in normal block flow and can surprise developers when spacing does not equal the sum they expected.

3. box-sizing: content-box vs. border-box

Value	How width/height are interpreted
content-box	The declared width/height applies to the content box. Padding and border are added outside it.
border-box	The declared width/height includes content, padding and border, making total sizing easier to control.

```
* { box-sizing: border-box; }
```

border-box is commonly used because an element declared as width: 300px remains 300px wide including its padding and border.

4. Width/Height, Min/Max Constraints and Overflow

- width and height specify dimensions of an element.
- min-width and min-height prevent an element from becoming smaller than specified limits.
- max-width and max-height prevent an element from becoming larger than specified limits.
- overflow controls what happens when content is larger than the available box.

overflow value	Behaviour
visible	Overflowing content remains visible; this is the usual initial behaviour.

hidden	Overflowing content is clipped.
scroll	Scrollbars are provided so overflow can be viewed.
auto	Scrollbars are added when needed.

5. display: block, inline, inline-block and none

Display Value	Behaviour
block	Creates a block box that normally starts on a new line and can accept width/height.
inline	Participates within a line; width and height do not behave like a normal block box.
inline-block	Flows inline but can be sized more like a block box.
none	Removes the element from the layout; it does not occupy space.

HTML elements have default display behaviour, but CSS can change it using the display property.

6. Borders, Shadows and Consistent Spacing

```
.panel {
  padding: 16px;
  margin-bottom: 16px;
  border: 1px solid #ccc;
  box-shadow: 0 2px 6px rgba(0,0,0,0.15);
}
```

Consistent spacing improves visual rhythm and maintainability. A project can use a small spacing scale, such as 4px, 8px, 16px and 24px, rather than choosing unrelated values for every component.

Quick Revision

- The box model consists of content, padding, border and margin.
- Adjacent vertical margins can sometimes collapse.
- content-box excludes padding and border from the declared width; border-box includes them.
- Min/max constraints control how far an element can shrink or grow.
- overflow controls content that exceeds the element's box.
- display determines whether an element behaves as block, inline, inline-block or is removed with none.

Viva / Concept Check

1. Name the four parts of the CSS box model from inside to outside.

2. What is the difference between content-box and border-box?

3. What does overflow: auto do?

4. How is inline-block different from inline?

5. What does display: none do to page layout?

Lecture #7 — CSS Positioning and Stacking (Important)

1. Normal Document Flow and Static Positioning

Normal document flow is the default way browsers arrange elements. Block elements normally appear one after another vertically, while inline content flows within lines. The default position value is static, so the element remains in normal flow and offset properties such as top and left do not reposition it in the normal static case.

2. Relative Positioning

With position: relative, an element remains in normal document flow, but it can be visually offset from its original position using top, right, bottom or left. Its original space is still preserved.

```
.badge {
  position: relative;
  top: 5px;
  left: 10px;
}
```

3. Absolute Positioning and the Containing Block

An absolutely positioned element is removed from normal flow and positioned using offsets relative to its containing block. A common pattern is to make a parent position: relative and a child position: absolute.

```
.card { position: relative; }
.close-button {
  position: absolute;
  top: 8px;
  right: 8px;
}
```

In this pattern, the close button is positioned relative to the card rather than participating in the card's normal content flow.

4. Fixed and Sticky Positioning

Position	Behaviour	Typical Use
fixed	Removed from normal flow and positioned relative to the viewport in common cases; remains in place while the page scrolls.	Floating action button or fixed header
sticky	Behaves according to normal flow until a scrolling threshold is reached, then sticks within its scrolling/containing context.	Section heading or sticky navigation

```
.topbar { position: sticky; top: 0; }
```

5. Offset Properties

The properties top, right, bottom and left are called offset or inset properties. Their effect depends on the position value. They are commonly used with relative, absolute, fixed and sticky positioning.

6. z-index and Stacking Contexts

When elements overlap, the browser determines which one is painted in front. The z-index property can influence stacking order for positioned elements and other elements that participate in stacking contexts. A larger z-index does not automatically place an element above every element on the page because stacking contexts create separate stacking groups.

```
.modal {
  position: fixed;
  z-index: 1000;
}
```

7. Floats and Clearing

The float property was historically used to place content, especially images, to the left or right so surrounding inline content could wrap around it. It was also used for page layouts before Flexbox and Grid became common.

```
img.cover { float: left; margin-right: 16px; }
```

The clear property controls whether an element may appear next to previously floated elements.

```
.next-section { clear: both; }
```

For modern page layout, Flexbox and Grid are generally more suitable than float-based layouts. Floats remain useful for content-wrapping cases such as text around an image.

8. Common Layout Pitfalls

- Using absolute positioning for an entire page can make layouts fragile because elements are removed from normal flow.
- Forgetting the containing block can cause an absolutely positioned element to appear in an unexpected place.
- Using large z-index values without understanding stacking contexts can make overlap problems difficult to debug.
- Floated children can affect the height/flow of a container unless the layout is handled appropriately.
- Fixed elements can cover content if sufficient page spacing is not provided.

Quick Revision

- Normal flow is the browser's default layout behaviour; static is the default position value.
- relative keeps an element's original space but allows visual offsets.
- absolute removes an element from normal flow and positions it relative to its containing block.
- fixed is commonly tied to the viewport; sticky changes behaviour when a scrolling threshold is reached.
- z-index influences overlapping order within stacking rules and stacking contexts.
- Floats are useful for content wrapping, while Flexbox and Grid are preferred for most modern layouts.

Viva / Concept Check

1. What is normal document flow?

2. What is the main difference between relative and absolute positioning?

3. Why is position: relative often placed on the parent of an absolutely positioned child?

4. What is the difference between fixed and sticky positioning?

5. Why may a very large z-index still fail to place an element above another element?

6. Why are Flexbox and Grid usually preferred over floats for modern layouts?

Lecture #8 — CSS3 Features (Excluded)

1. Rounded Corners, Gradients, Shadows and Opacity

Modern CSS provides visual effects that previously required images or extra markup. border-radius creates rounded corners, gradients generate smooth colour transitions, box-shadow/text-shadow add depth, and opacity controls the transparency of an element.

Feature	Example	Purpose
Rounded corners	border-radius: 12px;	Rounds the corners of a box.
Linear gradient	background: linear-gradient(to right, #fff, #ddd);	Creates a colour transition along a line.
Box shadow	box-shadow: 0 2px 8px rgba(0,0,0,.2);	Adds a shadow around an element.
Opacity	opacity: 0.7;	Makes the entire element partially transparent.

2. 2D and 3D Transforms

The transform property changes an element visually without changing normal document flow. Common 2D functions are translate(), rotate(), scale() and skew(). 3D transforms add a Z axis and functions such as rotateX(), rotateY() and translateZ().

```
.card:hover {
  transform: translateY(-4px) scale(1.02);
}
.flip { transform: perspective(600px) rotateY(20deg); }
```

3. Transitions and Keyframe Animations

A transition animates a property when its value changes, while a keyframe animation defines multiple stages and can run automatically or repeatedly.

```
.button { transition: background-color .3s, transform .3s; }
.button:hover { transform: scale(1.05); }

@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.08); }
  100% { transform: scale(1); }
}
.logo { animation: pulse 2s infinite; }
```

4. Web Fonts, Icon Fonts and CSS Custom Properties

Web fonts allow a site to use fonts supplied through web resources instead of relying only on fonts installed on the user device. Icon fonts represent icons as font glyphs, although SVG icon sets are also common. CSS custom properties (variables) store reusable values and are referenced with var().

```
:root {
  --primary: #78433b;
  --space: 16px;
}
.card { color: var(--primary); padding: var(--space); }
```

5. Media Queries and Feature Queries

Media queries apply CSS according to device or viewport characteristics. Feature queries using @supports apply rules only when the browser supports a particular CSS feature.

```
@media (max-width: 768px) {
  .sidebar { display: none; }
}
@supports (display: grid) {
  .books { display: grid; }
}
```

6. Vendor Prefixes and Cross-Browser Compatibility

Some browser engines historically required prefixes such as -webkit- or -moz- for experimental features. Modern projects usually rely on standards-based CSS and build tools such as Autoprefixer when prefixes are required. Developers should test important layouts in multiple browsers and provide fallbacks for unsupported features.

Quick Revision

- CSS3-style capabilities include rounded corners, gradients, shadows, transforms and animations.
- Transforms change visual presentation; transitions animate changes; keyframes define multi-stage animations.
- CSS custom properties improve reuse and consistency.
- Media queries respond to device/viewport conditions; @supports checks feature support.
- Cross-browser testing remains important even when using modern standards.

Viva / Concept Check

1. What is the difference between a transition and a keyframe animation?
2. Name four common transform functions.
3. What is the purpose of a CSS custom property?
4. How is @supports different from @media?
5. Why should vendor prefixes not be added blindly?

Lecture #9 — Layout Design using Flexbox and Grid (Important)

1. Flex Container, Flex Items and Axes

Flexbox is primarily a one-dimensional layout system. Setting display: flex on a parent creates a flex container; its direct children become flex items. Layout is described using a main axis and a cross axis. With flex-direction: row, the main axis is horizontal; with column, it is vertical.

2. Core Flexbox Properties

Property	Purpose	Example
flex-direction	Sets main-axis direction.	row, column
justify-content	Aligns/distributes items on main axis.	center, space-between
align-items	Aligns items on cross axis.	center, stretch
flex-wrap	Allows items to move to another line.	wrap
flex-grow	Controls use of extra space.	flex-grow: 1
flex-shrink	Controls shrinking when space is limited.	flex-shrink: 0
flex-basis	Sets initial main-size basis.	flex-basis: 250px

```
.toolbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
}
```

3. CSS Grid Container and Tracks

CSS Grid is a two-dimensional layout system because rows and columns can be controlled together. display: grid creates a grid container. grid-template-columns and grid-template-rows define tracks; fr represents a fraction of available space and gap controls spacing between tracks.

```
.books {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
```

4. Line Placement, Spanning and Named Areas

Grid items can be positioned using grid lines, can span multiple rows or columns, or can be assigned to named areas. This makes Grid suitable for page-level layouts where both dimensions matter.

```
.layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 220px 1fr;
}
header { grid-area: header; }
main { grid-area: main; }
```

5. Flexbox vs. Grid

Question	Flexbox	Grid
Main strength	One-dimensional alignment	Two-dimensional layout
Best example	Navigation bar / row of buttons	Dashboard / card matrix / page structure
Rows and columns together	Limited	Designed for both
Content-driven alignment	Very strong	Strong, but track-driven

A practical rule is to use Flexbox when arranging items mainly in one direction and Grid when rows and columns must be designed together. They are complementary and are often nested inside each other.

Quick Revision

- Flexbox uses a main axis and a cross axis and is mainly one-dimensional.
- justify-content works on the main axis; align-items works on the cross axis.
- Grid controls rows and columns together and supports fr, gap, spanning and named areas.
- Flexbox and Grid can be combined in the same interface.

Viva / Concept Check

1. Why is Flexbox called one-dimensional?
2. What changes when flex-direction becomes column?
3. What does 1fr mean in Grid?
4. How can a grid item span multiple columns?
5. Give one layout where Grid is more suitable than Flexbox.

Lecture #10 — Responsive Design and Framework Fundamentals (Important)

1. Responsive and Mobile-First Design

Responsive design allows one interface to adapt to different screen sizes. Mobile-first design starts with styles for smaller screens and progressively adds layout enhancements for wider screens, commonly using min-width breakpoints.

2. Viewport Meta Tag and Fluid Units

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

The viewport meta tag helps mobile browsers use the device width as the layout viewport. Fluid units reduce dependence on fixed pixel sizes: **%** is relative to a containing value, **rem** is relative to the root font size, **vw** is relative to viewport width and **vh** to viewport height.

3. Media Queries, Breakpoints and Responsive Images

```
img { max-width: 100%; height: auto; }

@media (min-width: 768px) {
  .content { display: grid; grid-template-columns: 2fr 1fr; }
}
```

Breakpoints should represent points where the layout needs to change rather than specific device models. Responsive images can scale with their container, while srcset and picture can provide different image resources when needed.

4. Component-Based vs. Utility-First Frameworks

Component-oriented frameworks such as Bootstrap provide ready-made components and a grid system. Utility-first frameworks such as Tailwind CSS provide small classes that are composed directly in markup or JSX. Both approaches can speed development when used consistently.

Approach	Typical Style	Example
Bootstrap	Predefined components + utilities	container, row, col-md-6, btn
Tailwind CSS	Utility-first classes	flex, grid, p-4, md:grid-cols-3

5. Tailwind CSS Responsive Variants

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  <div className="p-4 shadow rounded">Book 1</div>
  <div className="p-4 shadow rounded">Book 2</div>
  <div className="p-4 shadow rounded">Book 3</div>
</div>
```

In this example the layout has one column by default, two columns at the md breakpoint and three at lg. Tailwind variants such as md: and lg: apply utilities at configured breakpoints.

6. Custom Classes and Reusable UI Components

Framework utilities do not remove the need for good component design. Repeated interface patterns should be extracted into reusable React components, and custom CSS/classes can be added when a design cannot be expressed clearly with existing utilities.

Quick Revision

- Responsive design adapts one interface to multiple screen sizes.
- Mobile-first styling starts small and adds enhancements at larger breakpoints.
- The viewport meta tag is important for correct mobile layout.

- Bootstrap emphasizes components/grid; Tailwind emphasizes composable utility classes.
- Responsive variants make utilities conditional on breakpoints.

Viva / Concept Check

1. What does mobile-first mean?
2. Why is the viewport meta tag important?
3. What is the difference between rem and vw?
4. How does md:grid-cols-2 differ from grid-cols-1 in Tailwind?
5. Compare component-based and utility-first CSS frameworks.

Lecture #11 — Core JavaScript Concepts (ES6+): Functions, Closures and Objects

1. Role of JavaScript; Script Loading

JavaScript adds behaviour and programming logic to web pages and is also used on servers through runtimes such as Node.js. A script can be embedded or loaded from an external file. defer downloads in parallel and executes after HTML parsing; async executes when downloaded and does not preserve normal dependency order between independent scripts.

2. var, let and const; Data Types (What is Global Variable)

var is function-scoped, while let and const are block-scoped. const prevents reassignment of the variable binding, although objects stored in a const variable can still have their properties changed. Primitive values include string, number, boolean, null, undefined, bigint and symbol; objects and arrays are reference values.

Let yy = 40;

```
if (true) {
    var x = 10;
    let y = 20;

    console.log(x); // 10
    console.log(y); // 20

console.log(y);

}

console.log(yy); // 40
console.log(x); // 10
console.log(y); // Error: y is not defined
```

3. Operators, Coercion and Control Flow

JavaScript can perform implicit type coercion. == allows coercion in equality comparison, whereas === compares without converting operand types and is normally preferred. Control flow includes if/else, switch, for, while and related constructs.

```
const age = 20;
if (age >= 18) {
  console.log("Adult");
}
console.log(5 == "5");   // true
console.log(5 === "5");  // false
```

4. Scope, Hoisting and Temporal Dead Zone

Scope determines where a variable can be accessed. JavaScript has global, function and block scope. Declarations are processed before execution in ways commonly described as hoisting. let and const exist in their block before the declaration line but cannot be accessed there; this interval is called the temporal dead zone.

```
console.log(name);
var name = "Ali";
console.log(name);
```

OutPut

```
undefined
Ali
```

console.log(name);

let name = "Ali";

console.log(name);

OutPut

ReferenceError

5. Functions and Parameters

```
function add(a, b = 0) { return a + b; }
const multiply = function(a, b) { return a * b; };
const square = n => n * n;

function total(...values) {
  return values.reduce((sum, n) => sum + n, 0);
}
```

Default parameters provide fallback values, rest parameters collect remaining arguments into an array, and the spread operator expands iterable/object values. Destructuring extracts values from arrays or properties from objects.

6. this, call/apply/bind and Closures

The value of this depends on how a normal function is called. call invokes a function with a selected this value and separate arguments; apply is similar but accepts an argument array; bind returns a new function with this fixed. Arrow functions do not create their own this binding.

A closure occurs when a function retains access to variables from its lexical environment even after the outer function has finished. Closures are useful for private state, callbacks and function factories, but unnecessary captured data can increase memory use.

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const counter = makeCounter();
console.log(counter()); // 1
```

7. Objects, Arrays, Template Literals and Modules

```
const book = { title: "Web Technologies", price: 1200 };
const books = [book];
console.log(`${book.title}: Rs. ${book.price}`);

// math.js
export const add = (a, b) => a + b;
// app.js
import { add } from "./math.js";

Template Literals
let name = "Ahmed";
let message = "Hello " + name + "!";
```

```
console.log(message);
let name = "Ahmed";
let message = `Hello ${name}!`;
console.log(message);
```

Quick Revision

- let and const are block-scoped; var is function-scoped.
- === avoids the coercion performed by ==.
- Functions can be declarations, expressions or arrow functions.
- Closures retain access to their lexical environment.
- ES modules use export and import to organize code.

Viva / Concept Check

1. What is the difference between let and var?
2. Why is === generally preferred to ==?
3. What is the temporal dead zone?
4. What is a closure? Give one use.
5. How does an arrow function handle this differently from a normal function?

Lecture #12 — Important Built-in Methods and Array Processing (V.V Important)

1. Iteration: forEach and for...of

forEach executes a callback for each array element and is useful for side effects such as logging or updating external state. for...of is a loop syntax for iterating over values of iterable objects and supports break and continue.

```
const prices = [100, 200, 300];
// for (int x = )

prices.forEach(price => console.log(price));

for (const prices of prices) {
  if (p > 200) break;
}
```

2. map, filter and reduce

Method	Returns	Typical Use
map	A new transformed array	Convert book objects to titles
filter	A new array of matching elements	Keep books below a price
reduce	One accumulated result	Calculate total price

```
const books = [
  { title: "A", price: 500 },
  { title: "B", price: 900 },
];

const titles = books.map(b => b.title); // Arrow Functions
const cheap = books.filter(b => b.price < 800);
const total = books.reduce((sum, b) => sum + b.price, 0);
```

3. find, some, every and sort

find returns the first matching element, some checks whether at least one element matches, and every checks whether all elements match. sort rearranges an array in place, so a copied array should be sorted when the original order must be preserved. Numeric sorting normally needs a comparator.

```
const nums = [20, 3, 100];
const sorted = [...nums].sort((a, b) => a - b);
```

4. Frequently Used String and Object Methods

Useful string methods include trim, includes, startsWith, endsWith, toLowerCase, toUpperCase, replace and split. Common object utilities include Object.keys, Object.values, Object.entries and Object.assign. Modern code also uses spread syntax to copy or merge objects

Let myArray = [{id;1, name:'ABC'}, {id:2, name:'XYZ'} , {id:3,name1:'FFFF'}]

Console.log(Object.keys);

5. Method Chaining (Important)

Var array =[]

arrayClone = array

arrayClone

```
const result = books
  .filter(b => b.price >= 500)
  .map(b => b.title.toUpperCase())
  .sort();
```

Method chaining passes the result of one operation to the next. It can make data-processing pipelines readable, but very long chains should be split into named steps when clarity suffers.

6. Immutability, Functional Style and Common Mistakes

An immutable style creates new arrays or objects instead of changing existing values. This is especially useful in React because predictable state changes make rendering easier to reason about. Common mistakes include forgetting that sort and splice mutate arrays, using map only for side effects, and failing to return a value from a map callback.

Quick Revision

- forEach is useful for side effects; map creates transformed arrays.
- filter selects elements; reduce combines elements into one result.
- find returns a matching element; some/every return booleans.
- sort mutates the original array unless a copy is made first.
- Immutability is useful for predictable data flow and React state.

Viva / Concept Check

1. What is the difference between map and forEach?
2. When would you use filter?
3. Why does numeric sort often need a comparator?
4. What does reduce return?
5. Why can mutating an array directly be a problem in React?

Lecture #13 — DOM Manipulation and Event Handling

1. DOM Tree, Nodes and document

The Document Object Model (DOM) represents an HTML document as a tree of objects called nodes. JavaScript accesses the current page through the document object and can inspect or modify elements, attributes and text.

2. Selecting Elements

```
const title = document.getElementById("title");
const firstBook = document.querySelector(".book");
const allBooks = document.querySelectorAll(".book");
```

getElementById selects by id. querySelector returns the first match for a CSS selector, while querySelectorAll returns a static NodeList containing all matches.

3. Creating, Updating and Removing Nodes

```
const li = document.createElement("li");
li.textContent = "New Book";
document.querySelector("ul").append(li);
li.remove();
```

4. classList and Inline Styles

classList provides add, remove, toggle and contains operations for CSS classes. Inline styles can be changed through element.style, but class-based styling is usually easier to maintain for non-dynamic design rules.

```
card.classList.add("selected");
card.classList.toggle("hidden");
card.style.backgroundColor = "white";
```

5. Events, addEventListener and Event Object

```
button.addEventListener("click", event => {
  console.log(event.target);
});
```

An event represents an occurrence such as a click, input, submit or key press. addEventListener attaches a handler without placing JavaScript inside HTML attributes. The event object contains information about the event, including target and currentTarget.

6. Bubbling, Capturing and Event Delegation

During event propagation, capturing moves from ancestors toward the target and bubbling moves from the target back through ancestors. Event delegation attaches one listener to a parent and handles events from matching descendants, which is useful for dynamic lists.

```
list.addEventListener("click", e => {
  if (e.target.matches("button.delete")) {
    e.target.closest("li").remove();
  }
});
```

7. preventDefault and stopPropagation

preventDefault prevents a browser default action, such as form submission or link navigation. stopPropagation stops the event from continuing through the propagation path. They solve different problems and should not be used interchangeably.

Quick Revision

- The DOM represents the document as a tree of nodes.
- querySelector/querySelectorAll use CSS selector syntax.
- Nodes can be created, updated, appended and removed with JavaScript.
- Events propagate through capturing and bubbling phases.
- Event delegation handles descendant events through a common ancestor.

Viva / Concept Check

1. What is the DOM?
2. How does querySelector differ from querySelectorAll?
3. What information does event.target provide?
4. What is event delegation and why is it useful?
5. What is the difference between preventDefault and stopPropagation?

Lecture #14 — Regular Expressions and JSON (Excluded)

1. Regular Expression Basics

A regular expression (regex) describes a text pattern. In JavaScript a regex can be written as a literal such as /abc/ or created with RegExp. Character classes define allowed characters, quantifiers specify repetition and anchors match positions such as the start or end of input.

Syntax	Meaning	Example
[A-Z]	One uppercase letter	/[A-Z]/
\d	A digit	/\d/
+	One or more	/\d+/
*	Zero or more	/a*/
^	$ start/end anchors	/^book$/

2. Groups, Alternation, Backreferences and Flags

Parentheses create groups, | expresses alternatives and backreferences can match text captured by an earlier group. Common flags include g for global matching, i for case-insensitive matching and m for multiline anchor behaviour.

```
const re = /^(book|article)-\d+$/i;
console.log(re.test("Book-25")); // true
```

3. JavaScript Regex Methods

Method	Used On	Purpose
test	RegExp	Returns true/false for a match.
exec	RegExp	Returns match details or null.
match	String	Returns matching information.
replace	String	Replaces matched text.
split	String	Splits text using a string or regex separator.

4. Validation and Parsing of Form Data

Regex can support format checks for structured input, but validation should match the actual business rule and should not become unnecessarily complex. Browser/client validation improves feedback; server-side validation is still required because client checks can be bypassed.

5. JSON Syntax and Data Exchange

JSON (JavaScript Object Notation) is a text format for structured data exchange. JSON supports objects, arrays, strings, numbers, booleans and null. Property names and string values use double quotes. JSON does not support functions, comments or undefined values.

```
{
  "title": "Web Technologies",
  "price": 1200,
  "available": true
}
```

6. JSON.stringify and JSON.parse

```
const book = { title: "Web", price: 1200 };
const jsonText = JSON.stringify(book);
const restored = JSON.parse(jsonText);
```

JSON.stringify converts a JavaScript value into JSON text when representable; JSON.parse converts valid JSON text into a JavaScript value. JSON is commonly used in REST API request and response bodies.

Quick Revision

- Regex describes text patterns using characters, classes, quantifiers, groups and anchors.
- Flags such as g, i and m change matching behaviour.
- test, exec, match, replace and split are common regex-related methods.
- JSON is a language-independent text format widely used for API data.
- JSON.stringify serializes; JSON.parse deserializes.

Viva / Concept Check

1. What is the purpose of ^ and $ in regex?
2. What does the i flag do?
3. How is test different from match?
4. Name values that JSON can represent.
5. What is the difference between JSON.stringify and JSON.parse?

Lecture #15 — Asynchronous JavaScript: Promises and Fetch API (Important)

1. Synchronous vs. Asynchronous Execution

Synchronous code performs steps in sequence and a long-running operation can delay later work. Asynchronous APIs allow operations such as network requests or timers to complete later while JavaScript can continue other work. JavaScript runtimes coordinate callbacks through an event loop and queues.

2. Event Loop, Queues and Callback Hell

The event loop coordinates execution when the call stack becomes available. Promise reactions are handled through the microtask mechanism, while timers and many other callbacks are scheduled as tasks. Deeply nested dependent callbacks can create callback hell, making error handling and control flow difficult to read.

3. Promises: States and Chaining

A Promise represents a future result and has pending, fulfilled or rejected states. then handles successful results, catch handles rejection and finally runs cleanup logic regardless of outcome. Returning values or promises from then enables chaining.

```
fetch("/api/books")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log("Finished"));
```

4. Promise.all

Promise.all waits for all supplied promises to fulfill and returns their results in order. If one rejects, the combined promise rejects. It is useful when independent asynchronous operations can run concurrently.

5. async / await and Error Handling

```
async function loadBooks() {
  try {
    const response = await fetch("/api/books");
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}
```

An async function always returns a Promise. await pauses that async function until the awaited promise settles, without blocking the entire JavaScript runtime. try…catch provides readable error handling.

6. AJAX with Fetch API: GET and POST

```
Async function (){
const response = await fetch("/api/books", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "New Book" })
});
}
```

Fetch performs HTTP requests and returns a Promise for a Response object. Response bodies can be parsed with methods such as json() or text(). fetch does not reject merely because the server returns 404 or 500, so response.ok or status should be checked.

7. CORS Basics

Cross-Origin Resource Sharing (CORS) is an HTTP-based mechanism that allows a server to state which other origins may access its resources from browser scripts. It is enforced by browsers and configured by the server through response headers. CORS is not a replacement for authentication or authorization.

Quick Revision

- Asynchronous APIs allow work to complete later without blocking all JavaScript execution.
- Promises move through pending to fulfilled or rejected.
- async/await provides promise-based asynchronous code with sequential-looking syntax.
- Fetch supports HTTP requests; HTTP error statuses should be checked explicitly.
- CORS controls permitted cross-origin browser access through server policy.

Viva / Concept Check

1. What is the difference between synchronous and asynchronous execution?
2. What are the three states of a Promise?
3. What happens when one Promise in Promise.all rejects?
4. Why should response.ok be checked with fetch?
5. What problem does CORS address?

GET and POST are included but Server Side Code is Excluded

Lecture #16 — Introduction to Server-Side Programming

1. Client-Side vs. Server-Side Responsibilities

Client Side	Server Side
Displays UI and handles user interaction.	Processes requests and business rules.
Runs mainly in the browser.	Runs in a server/runtime environment.
Can perform immediate input feedback.	Must enforce trusted validation and authorization.
Calls APIs to obtain/store data.	Communicates with databases and other services.

A secure application does not trust client-side code for sensitive rules because browser code and requests can be modified by the user. Important validation, authorization and data operations must be enforced on the server.

2. Web Server, Application Server and Runtime Environment

A web server accepts HTTP requests and can serve static resources or forward requests to application code. An application server or web framework executes application logic. A runtime environment provides the platform required to execute code; Node.js is a JavaScript runtime commonly used for server-side applications.

3. Express.js Architecture: Event Loop and Non-Blocking I/O

Node.js uses an event-driven model and is designed to perform many I/O operations without blocking the main JavaScript execution thread while waiting for network, file or database operations. Express.js is a lightweight web framework built on Node.js that provides routing, middleware and HTTP utilities.

4. npm and Project Setup

```
mkdir bookstore-server
cd bookstore-server
npm init -y
npm install express
```

npm is the package manager commonly used with Node.js. package.json stores project metadata, scripts and dependency information. Installed packages are recorded as dependencies and normally placed under node_modules.

5. First Express Web Server

```
const express = require("express");
const app = express();
const PORT = 5000;

app.get("/", (req, res) => {
  res.send("Book Store Server is running");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

6. Ports and Basic Routes

A port identifies a network service on a host. During development an Express application may listen on a port such as 3000. A route combines an HTTP method and a path with handler logic. For example, GET /books can return a list of books, while POST /books can later be used to create a book.

```
app.use(express.json());

app.get("/books", (req, res) => {
  res.json([{ id: 1, title: "Web Technologies" }]);
});

app.post("/books", (req, res) => {
  res.status(201).json(req.body);
});
```

7. Request–Response Flow

A typical flow is: Browser/React client sends an HTTP request → web/application server receives it → Express selects a matching route and runs handler logic → the server may access data or services → an HTTP response is returned to the client. This extends the client-server and three-tier concepts introduced earlier in the course.

Quick Revision

- Client-side code focuses on browser UI; server-side code handles trusted application logic and data access.
- Node.js is a JavaScript runtime; Express.js is a web framework running on Node.js.
- Non-blocking I/O helps a server handle other work while waiting for I/O operations.
- npm manages packages and package.json records project configuration/dependencies.
- An Express route combines an HTTP method, path and handler.
- Ports identify services; a development server commonly listens on a configured port.

Viva / Concept Check

1. Why must important validation also occur on the server?
2. What is the difference between Node.js and Express.js?
3. What does non-blocking I/O mean?
4. What information is stored in package.json?
5. What are the three basic parts of an Express route?
6. What is the purpose of a port number?